Unit 1 –Class Fundas (Part 1)

📘 **Q-1: History of Java (Simple Answer)**

- Java is an **object-oriented programming language**.
- It was developed by **James Gosling** in the early 1990s.
- At first, Java was made for **electronic devices** like TV and set-top boxes.
- Before Java, **C++ was considered**, but it was not used because it needs more memory and uses pointers (complex).
- James Gosling and his team created a new language called **"Green Talk"**, which was later named **"Oak"**.
- The name **Oak** was changed to **Java** because the name Oak was already used by another company.
- The name **Java comes from coffee (espresso beans)**.
- Java was designed with important features:
    - Portable
    - Strong (robust)
    - Platform independent
    - High performance
    - Multithreading
- Today, Java is used in:
    - Internet programming
    - Mobile apps
    - Games
    - Business applications
- Many versions of Java were released like:
    - JDK Beta
    - JDK 1.0
    - JDK 1.1

- J2SE 1.3, 1.4

✅ Short Exam Tip (2–3 lines)

👉 Java is an object-oriented language developed by James Gosling in the 1990s.
👉 It was first called Oak and later renamed Java.
👉 It is widely used for web, mobile, and business applications.

📘 Q-2: Features of Java (Simple Answer)

Java has many important features:

1️⃣ Simple

- Java is easy to learn.
- It uses syntax like C/C++.
- It does not use pointers → makes it simple.

2️⃣ Security

- Java is very secure.
- It helps to create virus-free programs.
- Security features:
  - No pointers
  - Runs inside JVM (safe environment)

3️⃣ Portable

- Java program can run on any system.
- You can move code from one OS to another.
  👉 Example: Windows → Linux → Mac

4️⃣ Object-Oriented (OOP)

- Java follows OOP concepts.
- It makes programs easy to manage.

Main concepts:

- Class
- Object
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

5⃣Robust (Strong)

- Java is strong and reliable.
- Features:
    - Good memory management
    - No pointers
    - Automatic garbage collection (removes unused data)

6⃣Multithreading

- Java can do many tasks at the same time.
  👉 Example: Download + play music together
- Saves memory because threads share memory

7⃣Architecture Neutral

- Java works the same on all systems.
- Data types size is fixed.
  👉 Example: int is always 4 bytes

8⃣Interpreted

- Java is both compiled and interpreted.
  Steps:

1. Compiler → converts code to bytecode // a.java →javac a.java → a.class (bytecode)
2. JVM → converts bytecode to machine code // run “a.class” file when program will fun

---

9️⃣High Performance

- Java gives good performance.
- No pointers → faster execution
- Uses efficient memory system

🔟 Dynamic

- Java loads classes when needed.
- Supports dynamic memory and updates

✅ Short Exam Tip (Very Important)

👉 Java is simple, secure, portable, and object-oriented language.
👉 It supports multithreading, high performance, and platform independence.

📘 Q: Explain Java Editions (Simple Answer)

Java has **3 main editions**:

1️⃣Java Platform, Micro Edition (J2ME)

- J2ME stands for **Java 2 Platform Micro Edition**
- It is used for **small devices**

👉 Examples:

- Mobile phones (feature phones)
- Set-top boxes
- It was developed by **Sun Microsystems**

2️⃣ Java Platform, Enterprise Edition (J2EE)

- J2EE stands for **Java 2 Platform Enterprise Edition**
- It is used for **large applications**

👉 Examples:

- Banking systems
- Web applications
- Server-based applications
- It is mainly used in **companies (enterprise level)**

3️⃣ Java Platform, Standard Edition (J2SE)

- J2SE stands for **Java 2 Platform Standard Edition**
- It is used for **desktop applications**

👉 Types of apps:

- CUI (Command Line programs)
- GUI (Window-based apps)
- Applets

✅ Short Exam Tip

👉 J2ME → Mobile devices
👉 J2EE → Enterprise (large apps)
👉 J2SE → Desktop applications

📘 Q-3: JVM, JRE and JDK (Simple Answer)

🔷 JVM (Java Virtual Machine)

- JVM stands for **Java Virtual Machine**.
- It is the **main part of Java** that runs programs.

Functions:

- Loads and verifies Java bytecode
- Converts bytecode → machine code
- Executes the program

👉 Simple idea:

- Java code → compiled into bytecode → JVM runs it

🔷 **JRE (Java Runtime Environment)**

- JRE stands for **Java Runtime Environment**
- It is used to **run Java programs**

It includes:

- JVM
- Core libraries
- Supporting files

👉 Simple idea:

- JRE gives everything needed to **run** Java

🔷 **JDK (Java Development Kit)**

- JDK stands for **Java Development Kit**
- It is used to **develop + run Java programs**

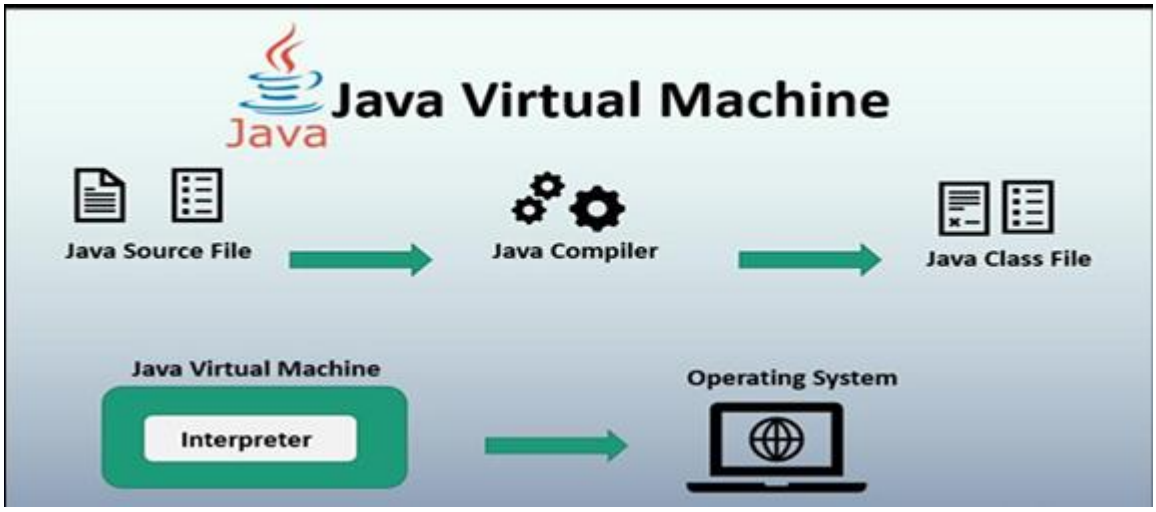
It includes:

- JRE
- Development tools

🔧 **Important JDK Tools**

1. **Appletviewer** → Runs applets without browser
2. **javac** → Converts source code → bytecode

3. **java** → Runs the program
4. **javadoc** → Creates documentation
5. **javah** → Creates header file
6. **javap** → Shows bytecode details
7. **jdb** → Debugging (find errors)



🔁 **Easy Comparison**

Component Use

JVM	Run program (execute bytecode)
JRE	Environment to run program
JDK	Develop + run program

✅ **Short Exam Tip**

👉 JVM executes Java program
👉 JRE provides environment
👉 JDK provides tools to develop Java

📘 **Q-4: Write a short note on compiling and executing basic java program.**

Ans: Steps:

Step 1: Save the program with .java extension (Note: Name of the program must be same as the name of the class)

Step 2: Open the command prompt and compile your program javac hello.java (where hello.java is the name of your file)

Step 3: Execute/run the program java hello Output: hello

📘 **Q-5 Write a short note on java IDE**

🔷 **What is IDE?**

- IDE stands for **Integrated Development Environment**
- It is a software used to **write, compile, run, and debug programs**

👉 Example: NetBeans, Eclipse

🔷 **NetBeans IDE**

- It supports languages like:
  - Java
  - PHP
  - C, C++

Features:

- Uses **javac compiler (from JDK)**
- Easy to develop Java applications
- Has built-in support for:
  - MySQL
  - Oracle database

- **Beginner friendly** (simple interface)
- Takes **more time to start**

🔷 **Eclipse IDE**

- It is an **open-source IDE**
- Supports many languages:
  - Java
  - C, C++
  - Python
  - PHP
  - JavaScript

Features:

- Uses its own compiler (not only javac)
- Shows more errors and warnings
- Fast loading (opens quickly)
- Needs extra tools for enterprise apps
- Has many windows for development
- Needs setup for database (JDBC)
- **Not beginner friendly** compared to NetBeans

🔁 **NetBeans vs Eclipse (Easy Table)**

Feature	NetBeans	Eclipse
Easy to use	Yes (Beginner)	No (Advanced)
Speed	Slow	Fast
Compiler	javac	Own compiler

Feature	NetBeans	Eclipse
Database support	Built-in	Need setup

✅ **Short Exam Tip**

👉 IDE is used to write and run Java programs
👉 NetBeans is easy for beginners
👉 Eclipse is faster and more advanced

📘 Q-6: Data Types in Java (Simple Answer)

🔷 What is Data Type?

- Data type means type of data stored in a variable
  👉 Example: number, decimal, character, true/false

🔷 Types of Data Types in Java

1️⃣ Integer Types (Whole Numbers)

- Used to store integer values (no decimal)
- Types:

Type	Size	Range
byte	8 bit	-128 to 127
short	16 bit	-32,768 to 32,767
int	32 bit	large values
long	64 bit	very large values

2️⃣ Floating Point Types (Decimal Numbers)

- Used for decimal values

Type Size Use

float 32 bit small decimal

double 64 bit large decimal

👉 Example: 3.14, 5.67

3️⃣ Character Type

- char is used to store a single character

Type Size

char 16 bit

👉 Example: 'A', 'B'

4️⃣ Boolean Type

- Used for true or false values

Type Value

boolean true / false

✅ Short Exam Tip

👉 Integer → whole numbers
👉 Float/Double → decimal numbers
👉 Char → character
👉 Boolean → true/false

📘 Q-7: What is ADT? (Simple Answer)

🔷 ADT (Abstract Data Type)

- ADT stands for Abstract Data Type

- It defines:
  👉 What operations to perform
  👉 Not how to perform

🔷 Key Points

- It does not show implementation details
- It gives only the idea or model
- It hides internal working

🔷 Examples

- List
- Stack
- Queue

🔷 Simple Understanding

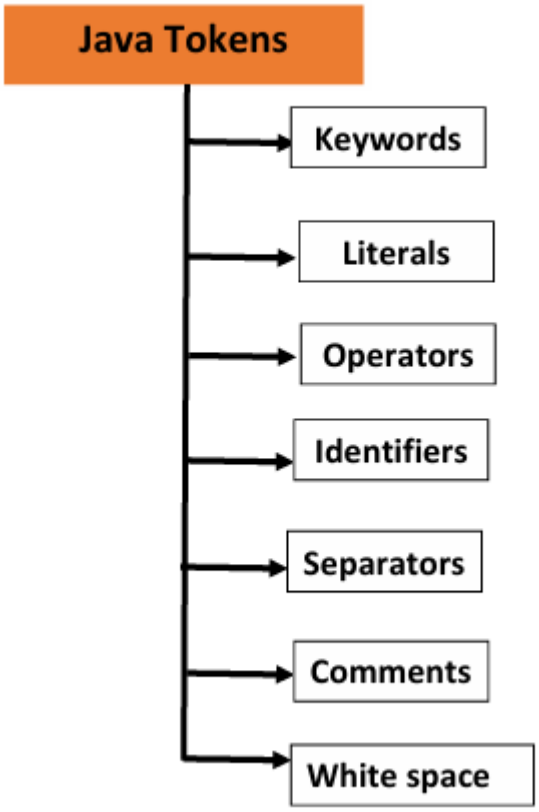
👉 ADT = what to do, not how to do

✅ Short Exam Tip

👉 ADT defines operations without implementation
👉 Examples: Stack, Queue, List

Q-8 Write a short note on java tokens

Ans:



Q-9 Write a short note on Operators in java

🔷 What is Token?

- Token is the smallest unit of a Java program
- It is identified by the Java compiler

👉 Example: keywords, variables, operators

🔷 Types of Java Tokens

Java has different types of tokens:

1️⃣ Keywords

- Keywords are reserved words
- Their meaning cannot be changed

👉 Examples:
int, class, if, else, enum

2️⃣ Literals

- Literals are constant values

Types:

- Integer → 10, 20
- Float → 2.5, 6.1
- String → "hello"

3️⃣ Operators

- Operators are symbols used for operations

👉 Examples:
+, -, *, /

4️⃣ Identifiers

- Identifiers are names of variables, classes, or functions

👉 Example:
int age; → age is identifier

5️⃣ Separators

- Used to separate parts of program

👉 Examples:
, ; ( ) { } [ ]

6️⃣ Comments

- Comments are not executed

- Used for explanation

Types:

- Single line → // comment
- Multi-line → /* comment */

7️⃣ Whitespace

- Used for spacing
- Includes:
  - Space
  - Tab
  - New line

✅ Short Exam Tip

👉 Token = smallest unit of program
👉 Types: Keyword, Literal, Operator, Identifier, Separator, Comment, Whitespace

🔷 **What is Operator?**

- An operator is a **symbol used to perform operations** on variables or values

---

1. Arithmetic Operators

- Used for **mathematical operations**

👉 Operators: + - * / %

✅ **Example:**

```
int a = 10, b = 5;

System.out.println(a + b); // 15
System.out.println(a - b); // 5
```

```
System.out.println(a * b); // 50
System.out.println(a / b); // 2
System.out.println(a % b); // 0
```

---

2. Relational Operators

- Used to **compare values**
- Output is **true or false**

👉 Operators: > < == != >= <=

✅ **Example:**

```
int a = 10, b = 5;

System.out.println(a > b);  // true
System.out.println(a < b);  // false
System.out.println(a == b); // false
System.out.println(a != b); // true
```

---

3. Logical Operators

- Used for **logical conditions**

👉 Operators: && || !

✅ **Example:**

```
int a = 10, b = 5;

System.out.println(a > 5 && b < 10); // true
System.out.println(a < 5 || b < 10); // true
System.out.println(!(a > b));       // false
```

---

4. Assignment Operators

- Used to **assign values**

👉 Operators: = += -= *= /=

✅ **Example:**

```
int a = 10;

a += 5; // a = a + 5
System.out.println(a); // 15
```

---

5. Conditional (Ternary) Operator

- Used as **short if-else**

👉 Syntax:

```
condition ? value1 : value2;
```

✅ **Example:**

```
int a = 10, b = 5;

int max = (a > b) ? a : b;
System.out.println(max); // 10
```

---

6. Increment / Decrement

- Used to **increase or decrease value by 1**

👉 Operators: ++ --

✅ **Example:**

```
int a = 5;

a++;
System.out.println(a); // 6

a--;
System.out.println(a); // 5
```

---

7. Bitwise Operators

- Used for **binary (bit-level) operations**

👉 Operators: & | ^ ~

✅ **Example:**

```
int a = 5, b = 3;

System.out.println(a & b); // 1
System.out.println(a | b); // 7
```

---

8. nstanceof Operator

- Checks object type

✅ **Example:**

```
String name = "Java";

System.out.println(name instanceof String); // true
```

<u>Or</u>

Syntax: objectname=classname Example: test t1=new test();

---

9. size of Operator:

ϖ It is used to find the size of the variable or the type.

Syntax: Datatype (SIZE) Example: Float(SIZE);

✅ **Short Exam Tip**

👉 Arithmetic → math
👉 Relational → compare
👉 Logical → conditions
👉 Assignment → store value

📘 **Q-10 Write a short note on Decision Control structures**

- Decision control structures are used to **take decisions in a program**.
- It checks a condition and executes code based on **true or false**.

👉 It helps to control the **flow of program**.

🔷 **Types of Decision Control Structures**

---

1 if Statement

- It executes block of code **only if condition is true**

✅ **Syntax:**

```
if(condition) {
   // code
}
```

✅ **Example:**

```
int a = 10;

if(a > 5) {
   System.out.println("a is greater");
}
```

---

2. if-else Statement

- It checks condition
- If true → run first block
- If false → run else block

✅ **Syntax:**

```
if(condition) {
   // true block
} else {
   // false block
}
```

✅ **Example:**

```
int a = 3;

if(a > 5) {
   System.out.println("a is greater");
} else {
   System.out.println("a is smaller");
}
```

---

3. else-if Ladder

- Used to check **multiple conditions**

✅ **Syntax:**

```
if(condition1) {
   // code
} else if(condition2) {
   // code
} else {
   // code
}
```

✅ **Example:**

```
int marks = 75;

if(marks >= 90) {
   System.out.println("A grade");
} else if(marks >= 60) {
   System.out.println("B grade");
} else {
   System.out.println("C grade");
}
```

---

4. switch Statement

- Used to select one option from many

✅ **Syntax:**

```
switch(value) {
   case 1:
      // code
      break;
   case 2:
      // code
      break;
   default:
      // code
}
```

✅ **Example:**

```
int day = 2;

switch(day) {
  case 1:
    System.out.println("Monday");
    break;
  case 2:
    System.out.println("Tuesday");
    break;
  default:
    System.out.println("Invalid");
}
```

✅ **Short Exam Tip**

👉 if → single condition
👉 if-else → two conditions
👉 else-if → multiple conditions
👉 switch → many options

📘 **Q-11: What is Loop? Types of Looping Structures**

🔷 **What is Loop?**

- A loop is used to **repeat a block of code again and again**
- It runs until a **condition becomes false**

👉 Example: printing numbers from 1 to 10

🔷 **Types of Looping Structures in Java**

1. **for loop**
2. **while loop**
3. **do-while loop**

---

1. for Loop

- Used when we know **how many times to repeat**

✅ **Syntax:**

```
for(initialization; condition; increment/decrement) {
   // code
}
```

✅ **Example:**

```
for(int i = 1; i <= 5; i++) {
   System.out.println(i);
}
```

👉 Output: 1 2 3 4 5

---

2. while Loop

- Used when we don't know exact number of times

✅ **Syntax:**

```
while(condition) {
   // code
}
```

✅ **Example:**

```
int i = 1;

while(i <= 5) {
   System.out.println(i);
   i++;
}
```

👉 Output: 1 2 3 4 5

---

3. do-while Loop

- It runs **at least one time** even if condition is false

✅ **Syntax:**

```
do {
   // code
} while(condition);
```

✅ **Example:**

```
int i = 1;

do {
   System.out.println(i);
   i++;
} while(i <= 5);
```

👉 Output: 1 2 3 4 5

🔁 **Difference Between Loops**

Loop	Condition Check	Use
for	Before loop	Known count
while	Before loop	Unknown count
do-while	After loop	Run at least once

✅ **Short Exam Tip**

👉 Loop is used to repeat code
👉 Types: for, while, do-while
👉 do-while runs at least one time

📘 **Q-12: Write a short note on jumping statements in java**

- Jumping statements are used to **transfer control of program from one place to another**
- They help to **change normal flow of execution**

🔷 **Types of Jumping Statements**

---

1) break Statement

- It is used to **stop the loop or switch statement immediately**

Example:

```
for(int i = 1; i <= 5; i++) {
  if(i == 3) {
    break;
  }
  System.out.println(i);
}
```

👉 Output: 1 2

---

2) continue Statement

- It is used to **skip current iteration and continue next loop**

Example:

```
for(int i = 1; i <= 5; i++) {
  if(i == 3) {
    continue;
  }
  System.out.println(i);
}
```

👉 Output: 1 2 4 5

---

3) return Statement

- It is used to **exit from method**
- It can also return a value

Example:

```
class Test {
  static int add(int a, int b) {
    return a + b;
  }

  public static void main(String args[]) {
    System.out.println(add(2,3));
  }
}
```

👉 Output: 5

✅ **Short Exam Tip**

👉 break → stop loop
👉 continue → skip iteration
👉 return → exit method

Q-13 Write a short note on typecasting

- **Typecasting means converting one data type into another data type**

🔷 **Types of Typecasting**

---

1) Implicit Typecasting (Automatic)

- **It is done automatically by Java**
- **Converts smaller data type into bigger data type**
- **No data loss**

Example:

```
int a = 10;
double b = a;   // int → double
System.out.println(b);  // 10.0
```

---

2) Explicit Typecasting (Manual)

- **It is done manually by programmer**
- **Converts bigger data type into smaller data type**
- **Data loss may happen**

Example:

```
double a = 10.5;
int b = (int)a;   // double → int
System.out.println(b);  // 10
```

🔷 **Important Points**

- **Implicit casting is safe**
- **Explicit casting may lose data (decimal part removed)**

✅ **Short Exam Tip**

👉 **Typecasting = changing data type**
👉 **Implicit → automatic**
👉 **Explicit → manual**

📘 **Q-14: Arrays in Java (Simple Answer)**

🔷 **What is Array?**

- **Array is a collection of elements of same data type**
- **All elements share same name**
- **Each element is accessed using index (position)**

👉 **Index starts from 0**

🔷 **Advantages of Array**

- **Code Optimization → Easy to store and manage data**
- **Random Access → Access any element using index**

🔷 **Disadvantages of Array**

- **Fixed Size → Size cannot change at runtime**

🔷 **Types of Array**

1. **One Dimensional Array (1-D)**
2. **Multi-Dimensional Array (2-D)**
3. **Jagged Array**

---

1) One Dimensional Array (1-D)

- **Array with only one row (single dimension)**

Syntax:

int a[] = new int[5];

Example:

```
import java.util.*;

class arr {
  public static void main(String args[]) {
    int i;
    int a[] = new int[5];
    Scanner sc = new Scanner(System.in);

    for(i = 0; i < 5; i++) {
      System.out.println("Enter array elements");
      a[i] = sc.nextInt();
    }

    for(i = 0; i < 5; i++) {
      System.out.println("Array ele " + a[i]);
    }
  }
}
```

---

2) Two Dimensional Array (2-D)

- **Stores data in rows and columns (matrix form)**
- **Total elements = rows × columns**

Syntax:

int a[][] = new int[2][3];

Declaration of array:

datatype[][] arrayname

OR

datatype arrayname[][]

Example:

```
import java.util.*;

class arr {
  public static void main(String args[]) {
    int i, j;
    int a[][] = new int[2][3];
    Scanner sc = new Scanner(System.in);

    for(i = 0; i < 2; i++) {
      for(j = 0; j < 3; j++) {
        System.out.println("Enter array elements");
        a[i][j] = sc.nextInt();
      }
    }

    for(i = 0; i < 2; i++) {
      for(j = 0; j < 3; j++) {
        System.out.println("Array ele " + a[i][j]);
      }
    }
  }
}
```

3) Jagged Array

- Array of arrays
- Row size is fixed, but column size is different

Example:

```
int a[][] = new int[2][];
a[0] = new int[3];
a[1] = new int[2];
```

Full Example:

```java
import java.util.*;

class arr {
  public static void main(String args[]) {
    int i, j;
    int a[][] = new int[2][];
    a[0] = new int[3];
    a[1] = new int[2];

    Scanner sc = new Scanner(System.in);

    for(i = 0; i < 2; i++) {
      for(j = 0; j < a[i].length; j++) {
        System.out.println("Enter array elements");
        a[i][j] = sc.nextInt();
      }
    }

    for(i = 0; i < 2; i++) {
      for(j = 0; j < a[i].length; j++) {
        System.out.println("Array ele " + a[i][j]);
      }
    }
  }
}
```

✅ **Short Exam Tip**

👉 **Array = collection of same data type**
👉 **Types: 1D, 2D, Jagged**
👉 **Index starts from 0**

📘 **Q-14: Command Line Arguments (Simple Answer)**

- **Command line argument is a value passed when running the Java program**
- **These values are given from the command prompt (console)**
- **They are used as input in the program**

👉 **Arguments are stored in args[] array**

🔷 **Key Points**

- **Arguments are passed at runtime**
- **Stored in String array (args[])**
- **Index starts from args[0]**

🔷 **Example:**

```
class cmd1 {
  public static void main(String args[]) {
    System.out.println(args[0]);
  }
}
```

🔷 **How to Run:**

```
javac cmd1.java
java cmd1 Hello
```

👉 **Output:**

```
Hello
```

🔷 **Explanation**

- **args[0] → first argument**
- **args[1] → second argument**

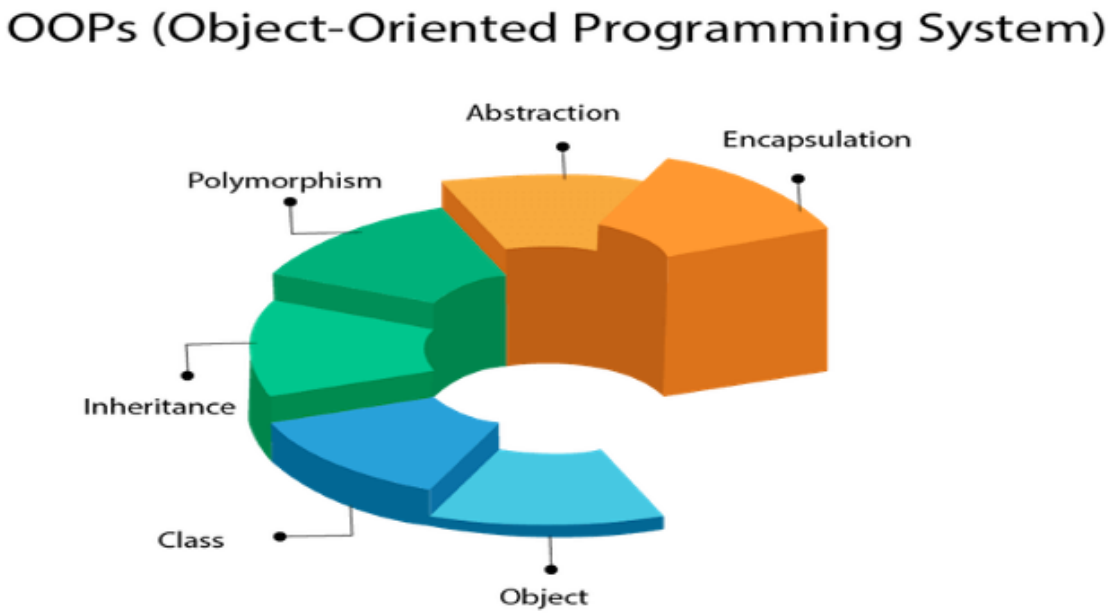
✅ **MCQ Points**

- **Arguments passed at runtime → Command line arguments**
- **Starting index → args[0]**

✅ **Short Exam Tip**

👉 **Command line arguments are inputs given at runtime**
👉 **Stored in args[] array**
👉 **Access using args[0], args[1]**

Unit 1 –Class Fundas (Part 2)



📘 **Q-1: Explain Class and Object**

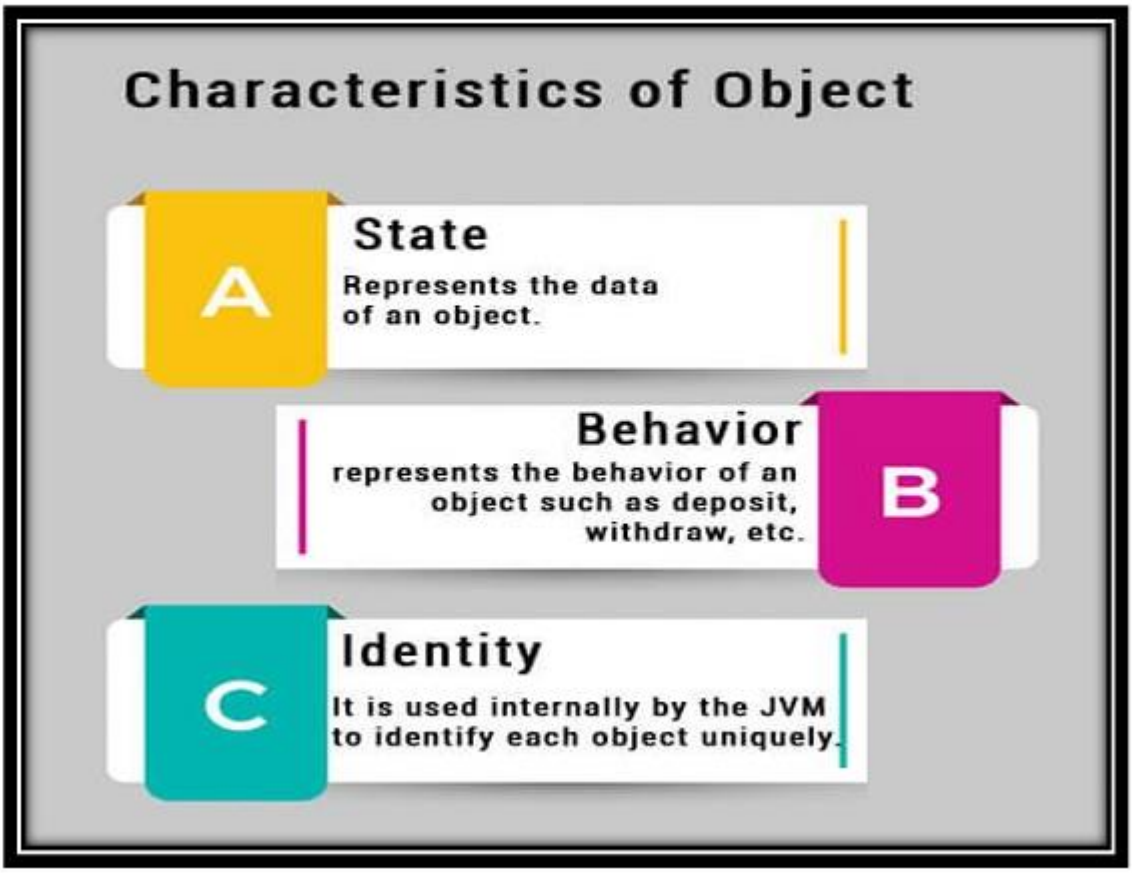
🔷 **Class**

- A class is a **template or blueprint** from which objects are created.
- It is a **logical entity** (not physical).
- A class defines **data (variables)** and **behavior (methods)**.
- When we create a class, we create a **new data type**.

🔷 **Object**

- An object is an **instance of a class**.
- It is a **real-world entity** like pen, car, bike, etc.
- Each object has its **own copy of variables**.

An object has three characteristics:



🔷 Key Points

- Class = group of objects having same properties
- Object = created from class
- Class defines structure, object uses it

🔷 Characteristics of Object

1. **State** → data (value of object)
2. **Behavior** → functionality (what object does)
3. **Identity** → unique identity of object

👉 Example:
Pen →

- State: color, name

- Behavior: writing

🔷 Syntax

```
class ClassName {
  int a;
  int b;

  void method() {
    // code
  }
}
```

🔷 Example

```
class student {
  int a = 10;
  int b = 20;

  void add() {
    System.out.println(a + b);
  }

  public static void main(String args[]) {
    student s = new student();
    s.add();
  }
}
```

🔷 MCQ Points

- Class → template
- Object → instance
- OOP main concept → class and object
- Object features → state, behavior, identity

✅ Final Exam Line (VERY IMPORTANT)

👉 A class is a blueprint and an object is its instance used to access data and methods.

MCQ		
Sr No.	Question	Answer
1	Which are the features or characteristics of object?	state,behavior,identity
2	A ..... is a template from which objects are created	class
3	.....and .....are the main concept of OOP	class, object

📘 Q-2: Explain Encapsulation

🔷 Definition

- Encapsulation is one of the **basic concepts of OOP**
- It means **wrapping data (variables) and methods together into one unit (class)**

🔷 Key Points

- In encapsulation, data is **hidden from outside classes**
- Data can be accessed **only through methods**
- It is also called **data hiding**

🔷 How to Achieve Encapsulation

- Declare variables as **private**
- Use **public getter and setter methods** to access data

🔷 Advantages of Encapsulation

- Data can be **secure (hidden)**
- We can control how data is used

- Variables can be **read-only or write-only**
- Improves **data safety and flexibility**

🔷 Example

```
class test {
  private int roll;

  public void get(int r) {
    roll = r;
  }

  public void set() {
    System.out.println(roll);
  }
}

class enc {
  public static void main(String args[]) {
    test t = new test();
    t.get(1);
    t.set();
  }
}
```

🔷 MCQ Points

- Encapsulation = data + methods together
- Also called → **data hiding**
- Use → private + getter/setter

✅ Final Exam Line

👉 Encapsulation is the process of wrapping data and methods into a single unit and hiding data using private access.

MCQ		
Sr No.	Question	Answer
1	Which are the features or characteristics of object?	state,behavior,identity
2	A ..... is a template from which objects are created	class
3	.....and .....are the main concept of OOP	class, object

📘 Q-3: Inheritance (Simple Answer)

🔷 Definition

- Inheritance is a mechanism where **one class gets properties and methods of another class**
- It is an important concept of **OOP**

🔷 Terms in Inheritance

1. **Class**

- A blueprint to create objects

2. **Super Class (Parent Class)**

- The class whose properties are inherited

3. **Sub Class (Child Class)**

- The class that inherits another class

🔷 Advantage of Inheritance

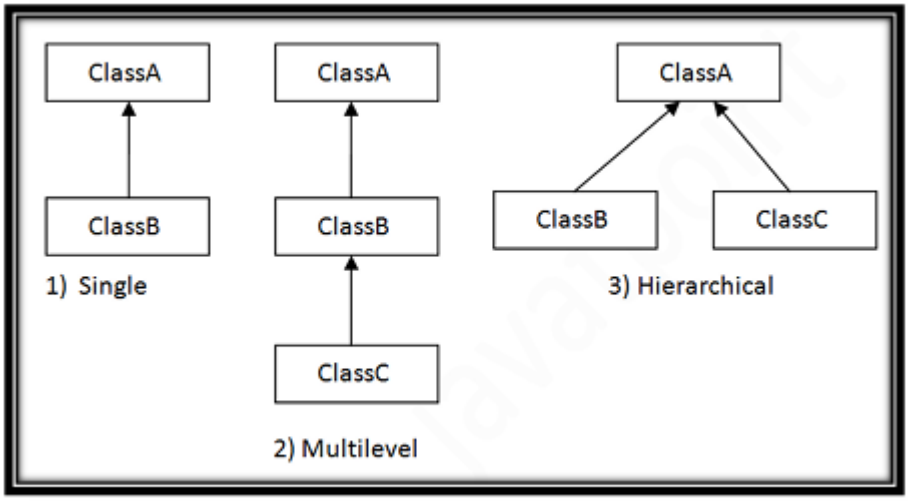
- **Reusability** → We can reuse code of existing class
- Saves time and reduces code

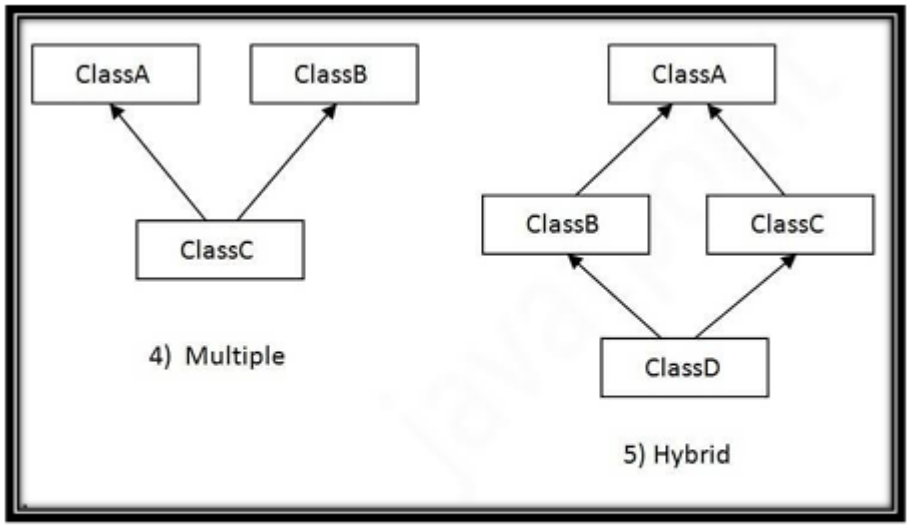
🔷 Syntax

```
class SubClass extends SuperClass {
   // code
}
```

👉 extends keyword is used for inheritance

🔷 Types of Inheritance





> **Note:** *Multiple inheritance is not supported in Java through class.*

1) Single Inheritance

- One class inherits one class

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class test {
  public static void main(String args[]) {
    Dog d = new Dog();
    d.bark();
    d.eat();
  }
}
```

---

2) Multilevel Inheritance

- A class inherits another class, and that class is inherited by another

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class BabyDog extends Dog {
  void sleep() {
```

```
    System.out.println("Sleeping");
  }
}

class test {
  public static void main(String args[]) {
    BabyDog b = new BabyDog();
    b.eat();
    b.bark();
    b.sleep();
  }
}
```

---

3) Hierarchical Inheritance

- Multiple classes inherit one parent class

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class Cat extends Animal {
  void sleep() {
    System.out.println("Sleeping");
  }
}

class test {
  public static void main(String args[]) {
    Dog d = new Dog();
    d.eat();
```

```
    d.bark();

    Cat c = new Cat();
    c.eat();
    c.sleep();
  }
}
```

---

4) Multiple Inheritance

- One class inherits **more than one class**
- ❌ Not supported in Java (with classes)
- Multiple inheritance is not supported in java by class. The reason is suppose A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

  Example:

```
class A {
   void msg() {
   System.out.println("Hello");
   }
}

class B {
    void msg() {
    System.out.println("Hi");
   }
}

class C extends A, B {
public static void main(String args[]) {
  C c1 = new C();
   c1.msg(); // Which msg() method will be called?
 }
}
```

Will generate compile time error

👉 Reason: **Confusion (ambiguity)**

---

5) Hybrid Inheritance

- Combination of more than one type of inheritance
- ❌ Not fully supported using classes
- Hybrid inheritance is the combination of more than 1 type of inheritance.

Example:

```
class Animal {
 void eat() {
    System.out.println("Eating");
 }
}
class Dog extends Animal { // Single level inheritance
 void bark() {
  System.out.println("Barking");
 }
}
class Cat extends Animal { // (This is also single-level, not multilevel)
  void sleep() {
   System.out.println("Sleeping");
  }
}
public class Main {
public static void main(String args[]) {
Cat c = new Cat();
c.eat();
c.sleep();
   Dog d = new Dog();
   d.bark();
   d.eat();
```

```
}
}
```

🔷 Important Points

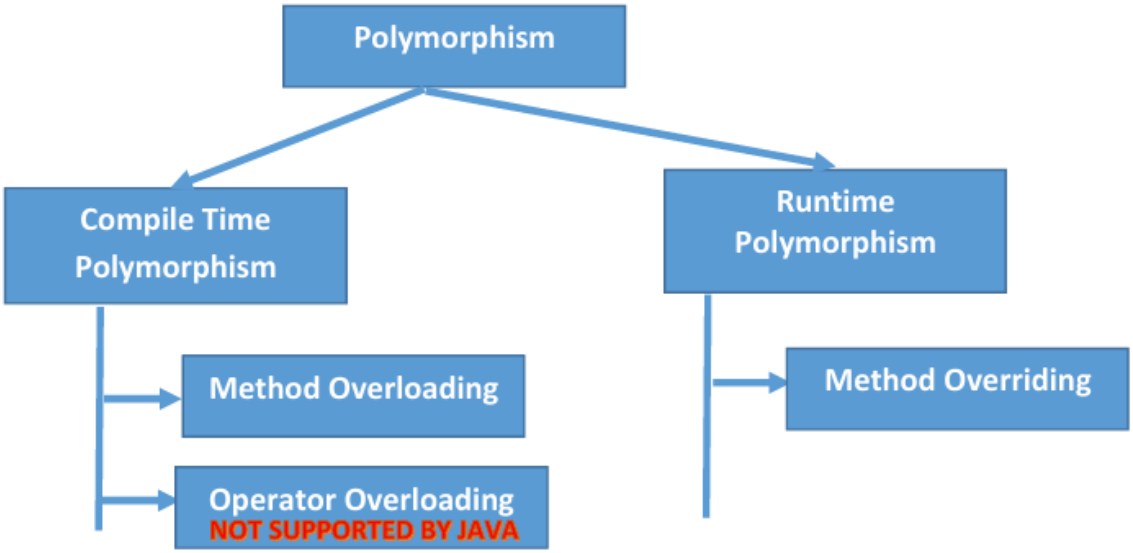
- Use extends keyword
- Java supports:
  - Single
  - Multilevel
  - Hierarchical
- Does NOT support:
  - Multiple (with classes)

✅ Final Exam Line

👉 Inheritance is a process where one class acquires properties and methods of another class for code reusability.

MCQ		
Sr No.	Question	Answer
1	The parent child concept is known as	Inheritance
2	The main advantage of inheritance is	reusability
3	Which inheritance is not supported in java?	Multiple Inheritance
4	In .....inheritance, there is one parent class and multiple child class and all child class access same parent class	Hierarchical
5	Which inheritance is known as chain of inheritance?	Multilevel
6	.....inheritance is the combination of more than one type of inheritance	Hybrid

📘 Q-3: Polymorphism (Simple Answer)



🔷 Definition

- Polymorphism means **one action in many forms**
- It comes from two words:
  - **Poly** → many
  - **Morph** → forms

👉 So, polymorphism means **many forms**

🔷 Types of Polymorphism

1. Compile Time Polymorphism
2. Run Time Polymorphism

🔷 1) Compile Time Polymorphism

- Also called **Static Polymorphism**
- Achieved by **Method Overloading**

🔶 Method Overloading

- Same method name but **different parameters**

- Can change:
    - Number of arguments
    - Type of arguments

Example:

```
class overload {
  void add(int a, int b, int c) {
    System.out.println(a + b + c);
  }

  void add(int a, int b) {
    System.out.println(a + b);
  }
}

class method1 {
  public static void main(String args[]) {
    overload o1 = new overload();
    o1.add(2, 3, 4);
    o1.add(1, 2);
  }
}
```

🔷 **2) Run Time Polymorphism**

- Also called **Dynamic Polymorphism**
- Achieved by **Method Overriding**

🔶 **Method Overriding**

- Same method in parent and child class
- Method call is decided at **runtime**

👉 Child class method is executed

Example:

```
class A {
  void run() {
    System.out.println("Hello");
  }
}

class B extends A {
  void run() {
    System.out.println("Hi");
  }
}

class sam {
  public static void main(String args[]) {
    B b1 = new B();
    b1.run();
  }
}
```

🔷 **Key Points**

- One method → many forms
- Overloading → compile time
- Overriding → run time

✅ **Final Exam Line**

👉 Polymorphism is a concept where one method can perform different actions in different situations.

Compile Time Polymorphism	Runtime polymorphism
1) This type of polymorphism is known as Static binding or early binding	1) This type of polymorphism is known as dynamic binding or late binding
2) It is achieved by method overloading as operator overloading is not supported by java	2) It is achieved by method overriding
3) Inheritance is not used in this type of inheritance	3) Inheritance is used in this type of polymorphism to achieve method overrding.

MCQ		
Sr No.	Question	Answer
1	Compile time polymorphism is also known as …….	Static binding
2	Runtime polymorphism is also known as….	Dynamic Binding
3	Compile time polymorphism is achieved by …….	Method overloading
4	The methods with same name but different parameters is known as ……..	Method overloading
5	Operator overloading is supported by java? (T/F)	False. Not supported
6	Runtime polymorphism is achieved by …..	Method overriding
7	In ……., child class overwrites the method of parent class	Method overriding

📘 Q-4: Constructor (Simple Answer)

🔷 Definition

- A constructor is a **special method**
- It has the **same name as the class**
- It is **automatically called when object is created**

🔷 Rules of Constructor

1. Constructor name must be **same as class name**
2. It has **no return type** (not even void)
3. It cannot be:

- abstract
- static
- final
- synchronized

🔷 Types of Constructor

1. Default Constructor
2. Parameterized Constructor

🔷 1) Default Constructor

- A constructor with **no parameters**

Example:

```
class A {
   A() {
      System.out.println("Default constructor");
   }
}

class test {
   public static void main(String args[]) {
      A a1 = new A();
   }
}
```

🔷 2) Parameterized Constructor

- A constructor with **parameters (arguments)**

Example:

```
class A {
   int roll;

   A(int r) {
      roll = r;
   }

   void display() {
      System.out.println(roll);
   }
}

class test {
   public static void main(String args[]) {
      A a1 = new A(10);
      a1.display();
   }
}
```

🔷 **Key Points**

- Constructor is used to **initialize object values**
- Called **automatically when object is created**
- Helps to set **initial data**

✅ **Final Exam Line**

👉 Constructor is a special method used to initialize objects and is automatically called when an object is created.

MCQ		
Sr No.	Question	Answer
1	Which method have same name as class name	Constructor
2	Constructor does not have return type (T/F)	True
3	The constructor with no or 0 arguments is known as……	Default constructor
4	The constructor with parameters is known as ……..	Parameterized constructor
5	When constructor is called?	When object of class is created.

Q-5 Write a short note on Constructor overloading

📘 **Q-5: Constructor Overloading (Simple Answer)**

🔷 **Definition**

- Constructor overloading means **having multiple constructors with same name but different parameters**
- It is used to **initialize objects in different ways**

🔷 **Key Points**

- Constructor name is **same as class name**
- Parameters must be **different (number or type)**
- Helps to create **different types of objects**

🔷 **Example**

```
class cons {
  int a, b;

  cons() {
    System.out.println("Hello");
  }

  cons(int a1) {
    a = a1;
  }

  cons(int a1, int b1) {
    a = a1;
    b = b1;
  }

  void display() {
    System.out.println(a);
    System.out.println(b);
  }
}

class consoverload {
  public static void main(String args[]) {
    cons c = new cons();
    cons c1 = new cons(10);
    cons c2 = new cons(10, 11);

    c1.display();
    c2.display();
  }
}
```

🔷 **Explanation**

- cons() → default constructor
- cons(int a1) → one parameter
- cons(int a1, int b1) → two parameters

👉 Same name but different parameters → **constructor overloading**

✅ **Final Exam Line**

👉 Constructor overloading is the process of defining multiple constructors with same name but different parameters in a class.

MCQ		
Sr No.	**Question**	**Answer**
1	**Constructor with same name but different parameters is known as .....**	**Constructor overloading**

📘 **Q-6: Static and Non-Static Members in Java**

🔷 **Static Variables**

- Static variable is declared using **static keyword**
- Only **one copy** is created
- It is **shared by all objects** of the class
- It is created at **class level**

🔷 **Example:**

```
class test {
  static int a = 20;

  void display() {
    a++;
    System.out.println(a);
  }
}

class sample {
  public static void main(String args[]) {
    test t1 = new test();
    test t2 = new test();
    test t3 = new test();

    t1.display();
```

```
    t2.display();
    t3.display();
  }
}
```

👉 Output will be:
21
22
23

👉 Same variable is shared by all objects

🔷 **Non-Static Variables**

- Non-static variable is declared **without static keyword**
- Each object has its **own separate copy**
- Values are **not shared between objects**

🔷 **Example:**

```
class test {
  int a = 20;

  void display() {
    a++;
    System.out.println(a);
  }
}

class sample {
  public static void main(String args[]) {
    test t1 = new test();
    test t2 = new test();
    test t3 = new test();

    t1.display();
    t2.display();
    t3.display();
```

```
  }
}
```

👉 Output will be:
21
21
21

👉 Each object has its own variable

🔷 **Key Difference**

Static	Non-Static
One copy	Multiple copies
Shared by all objects	Separate for each object
Class level	Object level

✅ **Final Exam Line**

👉 Static members are shared by all objects, while non-static members have separate copies for each object.

Static Members	Non-static Members
1) Static members can be accessed directly using classname	1) Non-static members can be accessed using object of class
2) Static variables can be accessed by static and non-static methods both	2) Non-static variables cannot be accessed inside static method
3) All the objects of class share same static variables	3) All the objects of the class have its own non-static variable.
4) Static variables are like global variables	4) Non-static variables are like local variables.
5) static keyword is used to declare the static members.	5) No keyword is required to declare non-static members.

MCQ		
Sr No.	Question	Answer
1	In .....variable, only 1 copy of variable is shared by all the objects of the class	static
2	Which keyword is used to declare static variable?	static
3	In .....variable, all the objects have their own copy of variable	Non-static
4	Static variables can be accessed by both .....and .....methods	static and non-static
5	Static variables are like .....variables	global
6.	Non-static variables are like .....variables	local

📘 Q-7: Varargs (Simple Answer)

🔷 Definition

- Varargs means **variable number of arguments**
- It allows a method to accept **zero or more values**
- Useful when we **don't know how many arguments** will be passed

---

🔷 Key Points

- Varargs uses **three dots (...)** after data type
- It treats arguments like an **array**
- We can pass:
  - No value
  - One value
  - Many values

🔷 Syntax

```
return_type method_name(data_type... variableName) {
   // code
}
```

🔷 Example

```
class varg {
   public void display(int... values) {
      System.out.println("hello");

      for(int i : values) {
         System.out.println(i);
      }
   }
}

class B {
   public static void main(String args[]) {
      varg a = new varg();

      a.display();           // no argument
      a.display(1, 2, 3, 4);   // multiple arguments
      a.display(12, 22);      // two arguments
   }
}
```

🔷 Explanation

- int... values → accepts multiple integers
- Works like an **array (values[])**
- Loop is used to print all values

✅ Final Exam Line

👉 Varargs allows a method to accept variable number of arguments using three dots (...) .

MCQ		
Sr No.	**Question**	**Answer**
1	If we don't know how many argument we will have to pass in the method, ... is the better approach	**varargs**
2	varargs uses	**ellipsis (...)**

📘 Q-8: IIB Block in Java (Simple Answer)

🔷 Definition

- IIB stands for **Instance Initializer Block**
- It is used to **initialize instance variables**

🔷 Key Points

- It runs **every time when object is created**
- It is written **inside class but outside methods**
- It executes **before constructor**

🔷 Example

```
class iib {
  int a;

  {
    a = 10;
  }

  public void display() {
    System.out.println(a);
```

```
  }
}

class im {
  public static void main(String args[]) {
    iib i = new iib();
    i.display();

    iib i2 = new iib();
    i2.display();
  }
}
```

🔷 **Explanation**

- Block { a = 10; } is **IIB**
- It runs **each time object is created**
- So both objects get value a = 10

✅ **Final Exam Line**

👉 IIB is used to initialize instance variables and it runs every time an object is created.

MCQ		
Sr No.	**Question**	**Answer**
1	IIB stands for	**Instance Intializer Block**
2	IIB runs each time the .....is created	**object of class**

End of Unit 1 ✅